

Model Training

Author: Yasithzz | 2026

01 -- Model Training

LifeMate Project -- English Documentation

1. What is Model Training?

Model training is the process of teaching a computer to make decisions based on examples. Think of how a student learns maths by studying many worked examples -- once enough examples are seen, the student can answer new questions correctly. In LifeMate, the RandomForestClassifier algorithm is trained on real lifestyle data from stress_dataset.csv. After training, the model can predict a user's stress level from their daily inputs.

2. train.py -- Line-by-Line Explanation

2.1 -- Docstring

```
"""
LifeMate Stress-Level Model Trainer
Trains a RandomForestClassifier on stress_dataset.csv.
Run: python train.py
Saves model/stress_model.pkl and model/model_meta.json.
"""
```

Describes the file's purpose. Running python train.py produces two files: the trained model binary (stress_model.pkl) and a JSON metadata file (model_meta.json).

2.2 -- Imports

```
import os, json, joblib
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report, accuracy_score
from imblearn.over_sampling import SMOTE
```

Import	Purpose
`os, json, joblib`	File management, JSON writing, model serialisation
`pandas` (`pd`)	Reading and handling the CSV dataset
`numpy` (`np`)	Numerical array operations
`RandomForestClassifier`	The core ML training algorithm
`train_test_split`	Splits data into train and test sets
`cross_val_score`	K-fold cross-validation for reliability
`accuracy_score`	Measures prediction accuracy
`SMOTE`	Balances unequal class distributions

2.3 -- Paths and Constants

```

BASE = os.path.dirname(__file__)
DATA = os.path.join(BASE, "..", "datasets", "stress_dataset.csv")
MODEL_DIR = os.path.join(BASE, "model")
os.makedirs(MODEL_DIR, exist_ok=True)

```

- * BASE -- absolute path of the directory containing train.py.
- * DATA -- path to stress_dataset.csv.
- * MODEL_DIR -- directory where model files are saved (ml-service/model/).
- * os.makedirs -- creates the model/ directory if it does not exist.

```

FEATURES = ["mood", "workload", "sleep_hours", "energy_level",
            "social_interaction", "exercise_done", "screen_time_hours", "water_cups"]
TARGET = "stress_level"
LABELS = ["Very Low", "Low", "Normal", "High", "Very High"]

```

- * FEATURES -- the 8 daily lifestyle columns the model learns from.
- * TARGET -- the column the model must predict.
- * LABELS -- the 5 human-readable stress class names (index 0-4).

2.4 -- load_data()

```

def load_data(path=DATA):
    df = pd.read_csv(path)
    X = df[FEATURES]
    y = df[TARGET]
    return X, y

```

Reads the CSV and separates it into X (input features) and y (target labels).

2.5 -- SMOTE Thresholds

```
MIN_SAMPLES_FOR_SMOTE = 30
MIN_SAMPLES_FOR_SPLIT = 10
```

- * SMOTE is applied only when there are at least 30 rows and each class has at least 6 samples.
- * A train/test split is done only when there are at least 10 rows.

2.6 -- train() Function

```
n_total = len(X)
class_counts = dict(zip(*np.unique(y, return_counts=True)))
min_class = min(class_counts.values())
```

- * n_total -- total number of dataset rows.
- * class_counts -- count of samples per stress class.
- * min_class -- size of the smallest class (used to decide SMOTE eligibility).

```
if n_total >= MIN_SAMPLES_FOR_SMOTE and min_class >= 6:
    smote = SMOTE(random_state=42)
    X_res, y_res = smote.fit_resample(X, y)
else:
    X_res, y_res = X.values, y.values
```

If enough data exists, SMOTE generates synthetic rows for minority classes so all classes are equally represented. Otherwise the raw data is used as-is.

```
X_train, X_test, y_train, y_test = train_test_split(
    X_res, y_res, test_size=0.2, random_state=42, stratify=y_res)
clf = RandomForestClassifier(
    n_estimators=200, max_depth=5, random_state=42, n_jobs=-1)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
acc = accuracy_score(y_test, y_pred)
```

- * 80% of data is used for training, 20% for testing.
- * RandomForestClassifier: 200 trees, maximum depth 5, all CPU cores used (n_jobs=-1).
- * clf.fit() -- the model learns from training data.
- * clf.predict() -- predicts on test data.
- * accuracy_score -- computes the proportion of correct predictions.

```
cv_folds = min(5, min_class) if min_class >= 2 else 0
if cv_folds >= 2:
    cv = cross_val_score(clf, X_res, y_res, cv=cv_folds, scoring="accuracy")
```

Cross-validation re-tests the model multiple times on different data splits to confirm consistent accuracy.

Fallback (too few samples):

```
clf = RandomForestClassifier(  
    n_estimators=100, max_depth=3, random_state=42, n_jobs=-1)  
clf.fit(X_res, y_res)
```

When fewer than 10 samples exist, a smaller model (100 trees, depth 3) is trained on all available data.

2.7 -- save()

```
joblib.dump(clf, model_path)  
json.dump({"features": FEATURES, "labels": LABELS, "accuracy": round(acc, 4)}, f, indent=2)
```

* stress_model.pkl -- serialised model binary loaded by app.py at runtime.

* model_meta.json -- features list, class labels, and accuracy -- consumed by the /model-info endpoint.

3. What is RandomForestClassifier?

A Random Forest builds an ensemble of decision trees, each trained on a random subset of the data and features. Each tree independently votes on the stress class; the class with the most votes wins. This makes the model robust to noise and outliers.

LifeMate settings:

Parameter	Value	Why
`n_estimators`	200	More trees -> more stable predictions
`max_depth`	5	Limits tree complexity, prevents overfitting
`random_state`	42	Reproducible results across runs
`n_jobs`	-1	Uses all CPU cores for faster training

4. What is SMOTE and Why?

SMOTE -- Synthetic Minority Over-sampling Technique.

If "Very High" stress records number only 5 while "Very Low" records number 100, a naive model will almost always predict "Very Low" because that dominates the data. SMOTE generates mathematically interpolated synthetic rows for the minority classes so every stress level is equally represented, forcing the model to learn genuine patterns for all 5 classes.

5. The 8 Features Explained

Feature	Meaning	Range
`mood`	How the user feels emotionally	1 (Stressed) -> 5 (Great)
`workload`	Perceived work pressure	1 (Very Light) -> 5 (Very Heavy)
`sleep_hours`	Hours slept last night	0 -> 12
`energy_level`	Physical energy today	1 (Exhausted) -> 5 (Very Energized)
`social_interaction`	Social engagement level	1 (None) -> 5 (Very Social)
`exercise_done`	Whether exercise was done	0 = No, 1 = Yes
`screen_time_hours`	Hours on phone/PC	0 -> 14
`water_cups`	Cups of water consumed	0 -> 16

Output (target): stress_level -- integer 0 (Very Low) through 4 (Very High).

Documentation -- LifeMate Project, Yasithzz, 2026